

ĐẠI HỌC QUỐC GIA TP.HCM
TRƯỜNG ĐẠI HỌC BÁCH KHOA



ĐẠI SỐ TUYẾN TÍNH

BÁO CÁO BÀI TẬP LỚN ĐỀ TÀI 6

Phép quay Given trong xử lý ảnh trên không gian
nhiều chiều

GVHD: NGUYỄN XUÂN MỸ
NHÓM 6

Danh sách thành viên:

STT	Họ và tên	MSSV	Hoàn thành (%)
1	Nguyễn Hữu Tây	2413095	100
2	Nguyễn Hữu Tuấn	2213785	100
3	Nguyễn Huỳnh Nhật Tuấn	2413761	100
4	Nguyễn Khắc Minh Tâm	2413064	100
5	Nguyễn Lâm Quang Thịnh	2413321	100
6	Nguyễn Lê Hoàng Long	2411923	100
7	Nguyễn Mạnh Hùng	2411321	100
8	Nguyễn Minh Đức	2111081	0
9	Nguyễn Minh Quân	2412911	100
10	Nguyễn Minh Thái	2313101	100

TP.HCM, 04/2025

Mục lục

1	Phần mở đầu	3
1.1	Đối tượng nghiên cứu	3
1.2	Phạm vi nghiên cứu	3
1.3	Mục tiêu nghiên cứu	3
1.4	Phương pháp nghiên cứu	3
1.5	Bố cục đề tài	3
2	Cơ sở lý thuyết của phép quay Given	4
2.1	Khái niệm phép quay Given	4
2.1.1	Định nghĩa	4
2.1.2	Tính chất toán học	4
2.2	Phương pháp tính toán phân tích phép quay	4
2.2.1	Phương Pháp Tính Góc Quay Givens	4
2.2.2	Phương Pháp Áp Dụng Givens Cho Ma Trận Lớn	4
2.2.3	Phương Pháp Tối Ưu Hóa Tính Toán	5
2.2.4	Phương Pháp Phân Tích Hiệu Quả	5
2.3	Ví dụ về phép quay	5
2.3.1	Triệt tiêu 1 phần tử trong vector	5
2.3.2	Triệt tiêu phần tử trong ma trận(ứng dụng trong phân rã QR) . . .	6
3	Các ứng dụng của phép quay Given	6
3.1	Các ứng dụng của phép quay trong xử lý hình ảnh	6
3.1.1	Trong không gian 2 chiều	6
3.1.2	Trong không gian 3 chiều	7
3.2	Các ứng dụng của phép quay trong không gian nhiều chiều	8
3.2.1	Phân rã QR	8
3.2.2	Xử lý tín hiệu số	8
3.2.3	Thị giác máy tính và mô hình hóa 3D	8
3.2.4	Tính toán song song và hiệu năng cao	8
3.2.5	Học máy và phân tích dữ liệu nhiều chiều	8
3.2.6	Hệ thống điều khiển và mô phỏng vật lý	8
4	Giới thiệu về Matlab	9
4.1	Tổng quan về Matlab	9
4.2	Một số lệnh Matlab cơ bản	9
5	Thuật toán của phép quay trong xử lý hình ảnh	9
5.1	Thuật toán trong không gian 2 chiều	9
5.1.1	Mô tả phép quay trong không gian hai chiều	9
5.1.2	Thuật toán xoay ma trận 2D in-place	10
5.1.3	Kết luận	11
5.2	Thuật toán trong không gian 3 chiều	11
5.2.1	Mục đích của thuật toán	11
5.2.2	Đầu vào	11
5.2.3	Tính toán hệ số quay	11
5.2.4	Tạo ma trận quay Given $G(i, j, c, s)$	11
5.2.5	Áp dụng phép quay	12



5.2.6	Kết quả	12
5.2.7	Đoạn code mẫu	12
5.3	Thuật toán Xoay ảnh bằng phép quay Givens trong xử lý ảnh số	12
5.3.1	Ý tưởng thuật toán	12
5.3.2	Đoạn code Matlab	13
5.3.3	Kết quả của đoạn code	14
5.3.4	Nhận xét	15
6	Lời kết	16
7	Tài liệu tham khảo	17

1 Phần mở đầu

1.1 Đối tượng nghiên cứu

Đề tài này tập trung nghiên cứu sâu về phép quay Givens - một kỹ thuật biến đổi trực giao trong đại số tuyến tính và cách để áp dụng nó vào việc xử lý hình ảnh số, đặc biệt trong các không gian nhiều chiều. Đối tượng chính bao gồm các ma trận và vector trong không gian n chiều và các ứng dụng cụ thể của phép quay này trong những bài toán xử lý hình ảnh.

1.2 Phạm vi nghiên cứu

Đề tài chủ yếu hướng tới phạm vi toán học của phép quay Givens và các thuật toán xử lý ảnh trong không gian, ứng dụng của phép quay trong xử lý hình ảnh như làm mịn ảnh, nén ảnh hay phân tích cấu trúc ảnh trong không gian đa chiều.

1.3 Mục tiêu nghiên cứu

Hiểu rõ định nghĩa, bản chất toán học và tính chất của phép quay Givens.

Tìm hiểu cách xây dựng ma trận Givens và cách nó được áp dụng để triệt tiêu phần tử trong ma trận.

Khảo sát các ứng dụng của phép quay Givens trong xử lý hình ảnh nhiều chiều.

Đưa ra các ví dụ minh họa cụ thể bằng hình ảnh và Matlab.

1.4 Phương pháp nghiên cứu

Tìm kiếm, thu thập và tổng hợp lý thuyết từ giáo trình đại số tuyến tính, các bài báo và các nghiên cứu liên quan đến đại số tuyến tính và xử lý ảnh.

Phân tích và trình bày lại lý thuyết một cách dễ hiểu và theo hệ thống logic.

Triển khai mô phỏng trên phần mềm Matlab để minh họa cho phép quay Givens.

Đánh giá hiệu quả của phương pháp thông qua thực nghiệm.

1.5 Bố cục đề tài

Đề tài được chia làm 4 phần chính:

- Phần 1 (Phần mở đầu): Giới thiệu tổng quan về đề tài và hướng nghiên cứu.
- Phần 2 (Cơ sở lý thuyết): Trình bày các cơ sở lý thuyết của phép quay Givens bao gồm khái niệm, phương pháp tính toán và các ví dụ minh họa.
- Phần 3 (Ứng dụng và các thuật toán): Phân tích thuật toán và ứng dụng phép quay Givens trong xử lý ảnh nhiều chiều.
- Phần 4 (Lời kết): Kết luận vấn đề và đưa ra hướng phát triển tiếp theo cho đề tài.

2 Cơ sở lý thuyết của phép quay Givens

2.1 Khái niệm phép quay Givens

2.1.1 Định nghĩa

Phép quay Givens là phép biến đổi trực giao cục bộ trong không gian $\mathbb{R}^n$, chỉ tác động lên 2 tọa độ (i, j) cố định.

Ma trận Givens $G(i, j, \theta) \in \mathbb{R}^{n \times n}$

$$G_{kk} = 1 \quad (\forall k \neq i, j), \quad G_{ii} = G_{jj} = \cos \theta, \quad G_{ij} = -\sin \theta, \quad G_{ji} = \sin \theta$$

Ví dụ: Trong $\mathbb{R}^4$, quay mặt phẳng (x_2, x_4) :

$$G(2, 4, \theta) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \theta & 0 & -\sin \theta \\ 0 & 0 & 1 & 0 \\ 0 & \sin \theta & 0 & \cos \theta \end{bmatrix}$$

2.1.2 Tính chất toán học

- **Bảo toàn chuẩn:** $\|Gx\| = \|x\|$.
- **Tính trực giao:** $G^T G = I_n$, $\det(G) = 1$.
- **Khả nghịch:** $G^{-1} = G^T = G(i, j, -\theta)$.
- **Giao hoán:** Tích các Givens không giao hoán (trừ khi cùng mặt phẳng quay).

2.2 Phương pháp tính toán phân tích phép quay

2.2.1 Phương Pháp Tính Góc Quay Givens

Mục tiêu: Cho vector $x = [x_i, x_j]^T$, tìm góc θ sao cho sau phép quay, thành phần thứ j bằng 0.

Công thức tính: $c = \cos(\theta)$ và $s = \sin(\theta)$

- Nếu $|x_j| > |x_i|$

$$\tau = \frac{-x_i}{x_j}, \quad s = \frac{1}{\sqrt{1 + \tau^2}}, \quad c = s \cdot \tau$$

- Ngược lại:

$$\tau = \frac{-x_j}{x_i}, \quad c = \frac{1}{\sqrt{1 + \tau^2}}, \quad s = c \cdot \tau$$

2.2.2 Phương Pháp Áp Dụng Givens Cho Ma Trận Lớn

Mục tiêu: Đưa ma trận A về dạng tam giác trên (ứng dụng trong phân rã QR).

Phương pháp giải:

- Chọn phần tử cần khử: Tìm $a_{ij} \neq 0$ (thường chọn phần tử dưới đường chéo).
- Tính góc Givens: Dùng công thức trên với vector $[a_{ij}, a_{ji}]^T$.
- Áp dụng phép quay:

- Nhân ma trận $G(i, j, \theta)$ vào bên trái A để khử a_{ji} .
- Nếu cần bảo toàn trị riêng, nhân thêm G^T vào bên phải.

2.2.3 Phương Pháp Tối Ưu Hóa Tính Toán

2.2.3.1 Tránh tính hàm lượng giác

Thay vì tính θ , dùng trực tiếp c và s từ:

$$r = \sqrt{x_i^2 + x_j^2}, \quad c = \frac{x_i}{r}, \quad s = -\frac{x_j}{r}$$

$\Rightarrow$ Giảm chi phí tính toán.

2.2.3.2 Cập nhật ma trận một phần

Thay vì nhân toàn bộ ma trận G , chỉ cập nhật 2 hàng/dòng liên quan:

$$\begin{cases} a'_{jk} = c \cdot a_{jk} - s \cdot a_{ik} \\ a'_{ik} = s \cdot a_{jk} + c \cdot a_{ik} \end{cases}$$

$\Rightarrow$ Độ phức tạp giảm từ $O(n^3)$ xuống $O(n^2)$.

2.2.4 Phương Pháp Phân Tích Hiệu Quả

2.2.4.1 Độ ổn định số học

Phép quay Givens ổn định hơn phép biến đổi Householder với ma trận thưa.
Không gây phóng đại sai số như phương pháp Gram-Schmidt.

2.2.4.2 So sánh với SVD và PCA

Ưu điểm:

- Givens hiệu quả khi chỉ cần khử một số phần tử (không cần phân tích toàn bộ ma trận).
- Phù hợp cho dữ liệu nhiều chiều thưa (sparse data).

Nhược điểm: Chậm hơn SVD/PCA nếu cần phân tích toàn bộ ma trận.

2.3 Ví dụ về phép quay

2.3.1 Triệt tiêu 1 phần tử trong vector

Cho $\mathbf{x} = \begin{bmatrix} 3 \\ 4 \end{bmatrix}^T$, ta muốn triệt tiêu $x_j = 4$:

Do $|4| > |3|$, tính:

$$\tau = \frac{3}{4}, \quad s = \frac{1}{\sqrt{1 + \left(\frac{3}{4}\right)^2}} = \frac{4}{5}, \quad c = \frac{4}{5} \cdot \left(-\frac{3}{4}\right) = -\frac{3}{5}$$

Ma trận Givens:

$$G = \begin{bmatrix} c & -s \\ s & c \end{bmatrix} = \begin{bmatrix} -\frac{3}{5} & -\frac{4}{5} \\ \frac{4}{5} & -\frac{3}{5} \end{bmatrix}$$

Áp dụng:

$$G\mathbf{x} = \begin{bmatrix} -\frac{3}{5} \cdot 3 + \left(-\frac{4}{5}\right) \cdot 4 \\ \frac{4}{5} \cdot 3 + \frac{3}{5} \cdot 4 \end{bmatrix} = \begin{bmatrix} -5 \\ 0 \end{bmatrix}$$

$\Rightarrow$ Thành công triệt tiêu $x_j = 4$

2.3.2 Triệt tiêu phần tử trong ma trận (ứng dụng trong phân rã QR)

Cho ma trận: $A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$

Muốn khử $a_{21} = 3$:

Tính góc Givens từ $[1, 3]^T$:

$$\tau = \frac{1}{3}, \quad c = \frac{3}{\sqrt{10}}, \quad s = \frac{1}{\sqrt{10}}$$

Ma trận Givens:

$$G = \begin{bmatrix} c & -s \\ s & c \end{bmatrix} = \begin{bmatrix} \frac{3}{\sqrt{10}} & -\frac{1}{\sqrt{10}} \\ \frac{1}{\sqrt{10}} & \frac{3}{\sqrt{10}} \end{bmatrix}$$

Áp dụng:

$$GA = \begin{bmatrix} \sqrt{10} & \frac{6+4}{\sqrt{10}} \\ 0 & \frac{2+12}{\sqrt{10}} \end{bmatrix}$$

$\Rightarrow$ Ma trận trở thành tam giác trên.

3 Các ứng dụng của phép quay Given

3.1 Các ứng dụng của phép quay trong xử lý hình ảnh

3.1.1 Trong không gian 2 chiều

3.1.1.1 Biến đổi hình ảnh và căn chỉnh ảnh:

Phép quay Givens được sử dụng để thực hiện các phép biến đổi trực giao, giúp căn chỉnh ảnh hay các thành phần trong ảnh một cách hợp lý và chính xác.

Điều này hữu ích trong các ứng dụng chỉnh sửa hay ghép ảnh (image stitching) và ổn định video, nơi cần căn chỉnh các khung hình hoặc ảnh chụp từ những góc độ khác nhau.

3.1.1.2 Nén và mã hóa ảnh

Trong các kỹ thuật nén ảnh, chẳng hạn như nén ảnh dựa trên biến đổi (transform-based compression), phép quay Givens có thể được sử dụng để thực hiện các phép biến đổi trực giao, giúp giảm thiểu sự dư thừa và tối ưu hóa quá trình mã hóa.

3.1.1.3 Phân tích thành phần chính (PCA)

PCA là một kỹ thuật giảm chiều dữ liệu quan trọng trong xử lý ảnh. Một trong các bước của PCA là tìm các vector riêng của ma trận hiệp phương sai của dữ liệu ảnh.

Phép quay Givens có thể được sử dụng trong quá trình phân tích thành phần chính (PCA) để thực hiện các phép biến đổi trực giao, giúp giảm chiều dữ liệu và trích xuất các đặc trưng quan trọng từ ảnh.

Phép quay Givens có thể được sử dụng trong các thuật toán lặp để tìm các vector riêng này một cách hiệu quả.

3.1.1.4 Xử lý tín hiệu và lọc ảnh

Trong xử lý tín hiệu và lọc ảnh, phép quay Givens có thể được sử dụng để thiết kế các bộ lọc trực giao, giúp loại bỏ nhiễu hoặc trích xuất các thành phần cụ thể từ ảnh.

3.1.1.5 Giải các bài toán tối ưu hóa

Trong một số bài toán tối ưu hóa liên quan đến xử lý ảnh, phép quay Givens có thể được sử dụng để xây dựng các thuật toán lặp nhằm tìm nghiệm tối ưu.

3.1.1.6 Một số ứng dụng khác

Nội suy ảnh: Phép quay Givens có thể được sử dụng trong các thuật toán nội suy để tính toán giá trị pixel tại các vị trí không nguyên sau khi thực hiện phép xoay hoặc biến đổi hình học khác.

Khử răng cưa (anti-aliasing): Khi xoay ảnh, các cạnh của đối tượng có thể bị răng cưa. Phép quay Givens có thể được kết hợp với các kỹ thuật khử răng cưa để tạo ra hình ảnh mượt mà hơn.

3.1.2 Trong không gian 3 chiều

3.1.2.1 Ước lượng và căn chỉnh hướng trong không gian 3D [2][3]

Trong các hệ thống thị giác máy tính 3D, đặc biệt là các ứng dụng như nhận diện khuôn mặt, mô hình hóa đầu người, hoặc theo dõi chuyển động, phép quay Givens được sử dụng để căn chỉnh góc nhìn và ước lượng tư thế đầu hoặc hướng nhìn.

Phép quay này giúp xoay tọa độ điểm ảnh sao cho các thành phần không mong muốn (thường là các phần tử ngoài đường chéo) bị triệt tiêu, từ đó dễ dàng trích xuất thông tin định hướng.

3.1.2.2 Phân rã QR trong phân tích ma trận ảnh 3D [4]

Phép quay Givens là một thành phần cốt lõi trong phân rã QR, một kỹ thuật quan trọng để giải các hệ phương trình tuyến tính trong các ứng dụng như:

- Tái tạo ảnh y tế 3D (CT, MRI).
- Nén và xử lý ma trận ảnh voxel.
- Giảm nhiễu và tách tín hiệu trong mô hình ảnh thể tích.

3.1.2.3 Chuẩn hóa không gian trong học sâu và thị giác 3D [5]

Trong các mô hình học sâu như PointNet hoặc các mạng 3D Convolutional Neural Networks (3D CNNs), phép quay Givens được sử dụng như một bước tiền xử lý để chuẩn hóa vị trí và hướng dữ liệu đầu vào. Nhờ tính chất giữ nguyên độ dài vector và tính trực giao, phép quay Givens giúp mô hình học tốt hơn các đặc trưng không gian trong dữ liệu 3D như đám mây điểm hoặc ảnh voxel.

3.1.2.4 Hiện thực hóa hiệu quả trên phần cứng [6]

Khi triển khai trên FPGA hoặc các vi mạch số, phép quay Givens thường được kết hợp với thuật toán CORDIC để tối ưu hóa hiệu suất tính toán lượng giác, vốn rất quan trọng trong các hệ thống nhúng xử lý ảnh 3D theo thời gian thực.

3.2 Các ứng dụng của phép quay trong không gian nhiều chiều

3.2.1 Phân rã QR

Phép quay Givens là công cụ hiệu quả để xây dựng phân rã QR , phân tích một ma trận thành tích của một ma trận trực giao Q và một ma trận tam giác trên R .

Phân rã QR đóng vai trò quan trọng trong các bài toán giải hệ phương trình tuyến tính, hệ quá xác định và bình phương tối thiểu.

Đặc biệt phù hợp với ma trận thưa, vì phép quay Givens chỉ triệt tiêu những phần tử nhất định mà không làm đầy ma trận.

3.2.2 Xử lý tín hiệu số

Được sử dụng trong các thuật toán như lọc tín hiệu, trích xuất đặc trưng và khôi phục tín hiệu.

Hữu ích trong các ứng dụng như radar, xử lý âm thanh và hình ảnh y tế.

3.2.3 Thị giác máy tính và mô hình hóa 3D

Dùng để xoay đối tượng hoặc dữ liệu trong mặt phẳng con, hỗ trợ các thao tác biến đổi tọa độ trong không gian 3D hoặc n chiều.

Giúp đơn giản hóa các phép tính, ví dụ như đưa vector về trục tọa độ, hoặc tạo góc nhìn phù hợp để phân tích mô hình.

Thường gặp trong mô phỏng vật lý, thiết kế 3D, và trực quan hóa dữ liệu.

3.2.4 Tính toán song song và hiệu năng cao

Mỗi phép quay chỉ tác động đến hai hàng hoặc hai cột, rất phù hợp với xử lý song song và các thuật toán phân tán.

Từng phép quay có thể thực hiện độc lập, cho phép tối ưu hóa tài nguyên tính toán.

Được sử dụng trong hệ thống lớn như siêu máy tính hoặc xử lý ma trận quy mô lớn.

3.2.5 Học máy và phân tích dữ liệu nhiều chiều

Giúp giảm tương quan giữa các đặc trưng, chuẩn hóa dữ liệu và làm mượt không gian đặc trưng trước khi áp dụng các thuật toán như PCA, SVD, LDA.

Cho phép xoay dữ liệu một cách chọn lọc để loại bỏ nhiễu hoặc đưa dữ liệu về dạng dễ xử lý hơn.

Đóng vai trò hỗ trợ trong giai đoạn tiền xử lý, đặc biệt khi làm việc với dữ liệu lớn và phức tạp.

3.2.6 Hệ thống điều khiển và mô phỏng vật lý

Giúp biến đổi hoặc giảm chiều các hệ phương trình trong các mô hình vật lý như chuyển động quay, dao động, hoặc mô phỏng vật thể.

Hữu ích trong việc phân tích chế độ riêng hoặc chuyển đổi giữa các hệ tọa độ.

Tăng tính trực quan và hiệu quả khi mô phỏng các hệ động lực học nhiều chiều.

4 Giới thiệu về Matlab

4.1 Tổng quan về Matlab

MATLAB (Matrix Laboratory) là một công cụ phổ biến trong các lĩnh vực như kỹ thuật, khoa học, và phân tích dữ liệu. MATLAB cung cấp các tính năng tương tác tuyệt vời cho phép người sử dụng thao tác dữ liệu linh hoạt dưới dạng mảng ma trận để tính toán và quan sát.

MATLAB là một ngôn ngữ lập trình và môi trường tính toán số học mạnh mẽ, được phát triển bởi MathWorks. MATLAB được sử dụng rộng rãi trong nhiều lĩnh vực như kỹ thuật, khoa học, phân tích dữ liệu, xử lý tín hiệu, điều khiển hệ thống, và trí tuệ nhân tạo. Nó cung cấp các công cụ tích hợp cho phép người dùng dễ dàng thực hiện tính toán số học, tạo hình ảnh trực quan, và lập trình các thuật toán phức tạp.

4.2 Một số lệnh Matlab cơ bản

Lệnh MATLAB	Chức năng
<code>uigetfile</code>	Mở hộp thoại để người dùng chọn tệp ảnh từ máy tính.
<code>imread</code>	Đọc ảnh từ tệp vào biến ma trận.
<code>size</code>	Lấy kích thước của ảnh (số dòng, cột, và kênh màu).
<code>mod</code>	Tính phần dư (thường dùng để kiểm tra số chẵn/lẻ).
<code>abs</code>	Tính trị tuyệt đối (dùng khi tính kích thước ảnh sau xoay).
<code>round</code>	Làm tròn số về số nguyên gần nhất.
<code>cos</code>	Tính cosin của góc (theo radian).
<code>sin</code>	Tính sin của góc (theo radian).
<code>zeros(..., 'uint8')</code>	Tạo ma trận ảnh toàn số 0 (đen) với kiểu dữ liệu uint8.
<code>figure</code>	Tạo cửa sổ đồ họa mới để hiển thị ảnh.
<code>subplot</code>	Chia cửa sổ hiển thị ảnh thành các vùng con để so sánh.
<code>imshow</code>	Hiển thị ảnh trong một cửa sổ đồ họa.
<code>title</code>	Đặt tiêu đề cho vùng hiển thị ảnh.
<code>num2str</code>	Chuyển số sang chuỗi (dùng để ghép tên tệp).
<code>imwrite</code>	Lưu ảnh ra tệp ảnh mới.
<code>disp</code>	Hiển thị thông báo hoặc chuỗi văn bản ra màn hình lệnh.

Bảng 1: Tổng hợp các lệnh MATLAB và chức năng sử dụng trong chương trình xoay ảnh

5 Thuật toán của phép quay trong xử lý hình ảnh

5.1 Thuật toán trong không gian 2 chiều

5.1.1 Mô tả phép quay trong không gian hai chiều

Phép quay là một phép biến đổi tuyến tính trong đại số tuyến tính, dùng để xoay các vector hoặc ma trận trong không gian Euclidean. Trong không gian 2 chiều, phép quay quanh gốc tọa độ với góc θ (theo radian) được biểu diễn bằng ma trận quay:

$$R(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}$$

Ma trận này có các đặc điểm sau:

- Là ma trận trực giao, nghĩa là $R^T R = I$ (với I là ma trận đơn vị)

- Định thức bằng 1, đảm bảo giữ nguyên diện tích và không có phản xạ

Khi áp dụng ma trận quay cho một vector tọa độ $\begin{bmatrix} x \\ y \end{bmatrix}$, ta được tọa độ mới:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = R(\theta) \cdot \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} x \cos \theta - y \sin \theta \\ x \sin \theta + y \cos \theta \end{bmatrix}$$

Phép quay có ứng dụng thực tiễn trong nhiều trường hợp như:

- **Căn chỉnh và đăng ký hình ảnh:** Khi cần so sánh hoặc kết hợp các hình ảnh chụp từ các góc nhìn khác nhau.
- **Nhận dạng và theo dõi đối tượng:** Giúp nhận diện đối tượng ở các góc độ khác nhau.
- **Tăng cường dữ liệu:** Tạo ra các biến thể của dữ liệu huấn luyện trong học máy.
- **Chỉnh sửa hướng hình ảnh:** Sửa các hình ảnh bị lệch hướng.

5.1.2 Thuật toán xoay ma trận 2D in-place

5.1.2.1 Cách 1: Sử dụng vòng lặp (Cycles)

Ý tưởng: Ma trận vuông được chia thành các "vòng" và mỗi vòng được xử lý bằng cách hoán đổi các phần tử theo nhóm bốn.

```
1 function rotateMatrix(matrix, n):  
2     for i from 0 to n/2 - 1:  
3         for j from i to n - i - 2:  
4             temp = matrix[i][j]  
5             matrix[i][j] = matrix[n-1-j][i]  
6             matrix[n-1-j][i] = matrix[n-1-i][n-1-j]  
7             matrix[n-1-i][n-1-j] = matrix[j][n-1-i]
```

5.1.2.2 Cách 2: Chuyển vị và đảo ngược

Ý tưởng: Thực hiện qua hai bước:

1. Chuyển vị ma trận
2. Đảo ngược thứ tự các phần tử trong mỗi hàng

```
1 function rotateMatrix(matrix, n):  
2     // Bước 1: Chuyển vị ma trận  
3     for i from 0 to n-1:  
4         for j from i+1 to n-1:  
5             swap(matrix[i][j], matrix[j][i])  
6  
7     // Bước 2: Đảo ngược mỗi hàng  
8     for i from 0 to n-1:  
9         for j from 0 to n/2 - 1:  
10            swap(matrix[i][j], matrix[i][n-1-j])  
11
```

So sánh hai cách tiếp cận

Tiêu chí	Cách 1: Vòng lặp	Cách 2: Chuyển vị và đảo ngược
Ý tưởng	Hoán đổi nhóm 4 phần tử	Chuyển vị rồi đảo ngược hàng
Độ phức tạp	$O(n^2)$	$O(n^2)$
Ưu điểm	Trực quan	Dễ triển khai
Nhược điểm	Tọa độ phức tạp	Yêu cầu hai bước

5.1.3 Kết luận

Phép quay trong không gian 2D là một ứng dụng quan trọng của đại số tuyến tính trong xử lý hình ảnh. Các thuật toán in-place như hoán đổi theo vòng lặp hoặc chuyển vị và đảo ngược cho phép xoay ma trận hiệu quả mà không cần thêm bộ nhớ.

5.2 Thuật toán trong không gian 3 chiều

5.2.1 Mục đích của thuật toán

Phép quay Givens được sử dụng để triệt tiêu một thành phần cụ thể trong vector hoặc ma trận 3D. Đây là công cụ quan trọng trong nhiều ứng dụng:

- Ước lượng tư thế đầu (Head Pose Estimation)
- Chuẩn hóa dữ liệu 3D (dùng trong PointNet, 3D CNN, v.v.)
- Phân rã QR trong xử lý ảnh voxel
- Tối ưu hóa phần cứng sử dụng thuật toán CORDIC

5.2.2 Đầu vào

Ma trận hoặc vector A có kích thước $n \times n$ hoặc $n \times 1$

Hai chỉ số hàng i và j cần thực hiện phép quay để triệt tiêu phần tử $A[j, k]$

Chỉ số cột k xác định vị trí phần tử cần triệt tiêu

5.2.3 Tính toán hệ số quay

Cho hai phần tử:

$$a = A[i, k] \quad \text{và} \quad b = A[j, k]$$

Tính toán các hệ số:

$$r = \sqrt{a^2 + b^2}, \quad c = \frac{a}{r}, \quad s = \frac{b}{r}$$

5.2.4 Tạo ma trận quay Given $G(i, j, c, s)$

G là ma trận đơn vị kích thước $n \times n$, trong đó các phần tử thay đổi tại các vị trí:

$$\begin{cases} G[i, i] = c \\ G[i, j] = s \\ G[j, i] = -s \\ G[j, j] = c \end{cases}$$

5.2.5 Áp dụng phép quay

Tùy theo việc triệt tiêu phần tử theo hàng hay cột:

- Tùy theo việc triệt tiêu phần tử theo hàng hay cột: $A' = G \times A$
- Nếu quay theo cột: $A' = A \times G^T$

5.2.6 Kết quả

Ma trận sau khi áp dụng phép quay Givens sẽ có phần tử $A[j, k] = 0$. Nếu áp dụng nhiều lần, ta thu được phân rã QR:

$$A = Q \times R$$

5.2.7 Đoạn code mẫu

```
1 import numpy as np
2
3 def givens_rotation(a, b):
4     if b == 0:
5         return 1.0, 0.0
6     r = np.hypot(a, b)
7     c = a / r
8     s = b / r
9     return c, s
10
11 def apply_givens_rotation(A, i, j, k):
12     a = A[i, k]
13     b = A[j, k]
14     c, s = givens_rotation(a, b)
15     G = np.identity(A.shape[0])
16     G[[i, i, j, j], [i, j, i, j]] = c, s, -s, c
17     A_rotated = G @ A # Quay theo hàng
18     return A_rotated, G
19
```

5.3 Thuật toán Xoay ảnh bằng phép quay Givens trong xử lý ảnh số

5.3.1 Ý tưởng thuật toán

Phép quay Givens (Givens Rotation): Sử dụng ma trận quay 2 chiều:

$$G = \begin{bmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{bmatrix}$$

để xoay từng pixel quanh tâm ảnh ban đầu.

Biến đổi tọa độ: Từng pixel $[n, d]$ được quy về tọa độ tương đối so với tâm ảnh, xoay bằng ma trận G , sau đó dịch về tâm ảnh mới để xác định vị trí trong ảnh kết quả. Với mỗi pixel (n, d) trong ảnh gốc, thực hiện các bước:

1. Tính tọa độ tương đối so với tâm ảnh.
2. Áp dụng ma trận quay Givens để xoay tọa độ.
3. Dịch ngược về tọa độ gốc trong ảnh mới.

4. Làm tròn và gán giá trị pixel từ ảnh gốc vào ảnh kết quả.

Làm tròn: Áp dụng sau khi nhân ma trận để đưa về chỉ số nguyên, vì ảnh số không chấp nhận chỉ số thực.

5.3.2 Đoạn code Matlab

Hàm main cho phép người dùng nhập góc xoay ảnh và gọi hàm xoay ảnh `rotateImage()`

```
1 function main()
2     degree = input('Chọn góc xoay: ');
3     rotated = rotateImage(degree);
4 end
```

Hàm `rotateImage()` với đầu vào là degree cho phép người dùng chọn ảnh, hàm sẽ thực hiện xoay theo góc mà người dùng đã nhập từ trước.

```
1 % Hàm xoay ảnh
2 function rotated_image = rotateImage(degree)
3     % Chọn ảnh
4     file = uigetfile('*.jpg;*.png;*.jpeg;*.gif', 'Chọn ảnh để xoay');
5     while file == 0
6         file = uigetfile('*.jpg;*.png;*.jpeg;*.gif', 'Bạn vẫn chưa chọn ảnh');
7     end
8     % Đọc hình ảnh
9     anh = imread(file);
10    % Góc xoay chuyển sang radian
11    theta = degree * pi / 180;
12    % Lấy kích thước ảnh ban đầu
13    [d0, n0, numberOfChannels] = size(anh);
14
15    % Tính toán tọa độ tâm ảnh gốc
16    if mod(d0, 2) == 0
17        b = d0/2;
18    else
19        b = (d0+1)/2;
20    end
21
22    if mod(n0, 2) == 0
23        a = n0/2;
24    else
25        a = (n0+1)/2;
26    end
27
28    O = [a; b]; % Tâm ảnh gốc
29
30    % Tính kích thước ảnh mới sau khi xoay
31    B = [abs(cos(theta)) abs(sin(theta)); abs(sin(theta)) abs(cos(theta))];
32    S = round(B * [n0; d0]);
33
34    % Tính tọa độ tâm ảnh mới
35    if mod(S(1,1), 2) == 0
36        as = S(1,1)/2;
37    else
38        as = (S(1,1)+1)/2;
39    end
40
41    if mod(S(2,1), 2) == 0
42        bs = S(2,1)/2;
43    else
44        bs = (S(2,1)+1)/2;
45    end
46
```

```

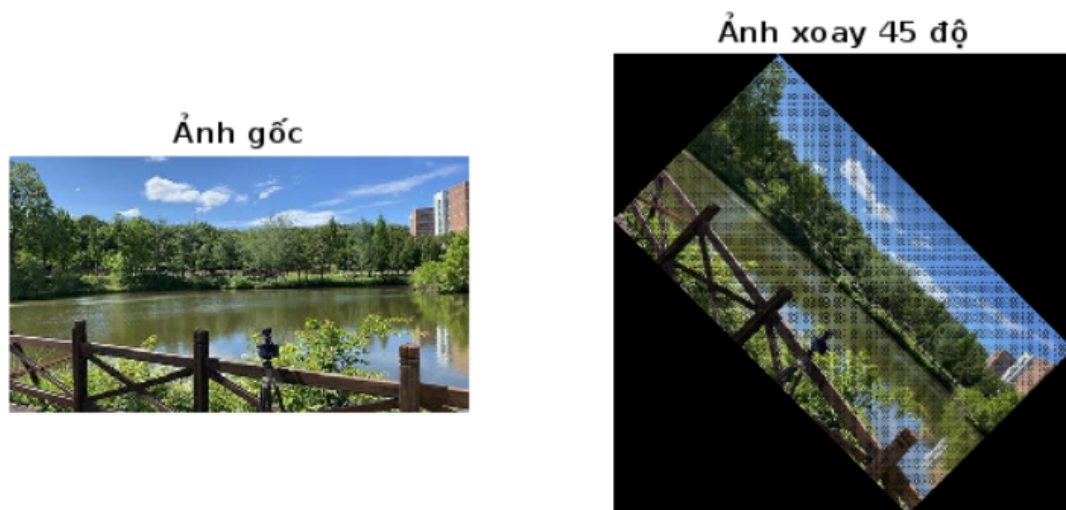
47 Os = [as; bs]; % Tâm ảnh mới
48
49 % Tạo khung ảnh mới với nền màu đen
50 khung = zeros(S(2,1), S(1,1), numberOfChannels, 'uint8');
51
52 % Ma trận phép quay Givens
53 G = [cos(theta) -sin(theta); sin(theta) cos(theta)];
54
55 % Thực hiện xoay từng pixel
56 for d = 1:d0
57     for n = 1:n0
58         % Tọa độ pixel tương đối so với tâm ảnh gốc
59         X0 = [n; d] - 0;
60
61         % Tọa độ sau khi xoay (tương đối so với tâm ảnh mới)
62         X = round(G * X0) + Os;
63
64         % Kiểm tra xem tọa độ mới có nằm trong ảnh kết quả không
65         if X(1,1) > 0 && X(1,1) <= S(1,1) && X(2,1) > 0 && X(2,1) <= S(2,1)
66             % Gán giá trị pixel từ ảnh gốc sang ảnh mới
67             khung(X(2,1), X(1,1), :) = anh(d, n, :);
68         end
69     end
70 end
71
72 % Trả về ảnh đã xoay
73 rotated_image = khung;
74
75 % Hiển thị kết quả
76 figure;
77 subplot(1, 2, 1);
78 imshow(anh);
79 title('Ảnh gốc');
80
81 subplot(1, 2, 2);
82 imshow(khung);
83 title(['Ảnh xoay ' num2str(degree) ' độ']);
84
85 % Lưu ảnh kết quả
86 imwrite(khung, ['rotated_' num2str(degree) '_degrees.jpg']);
87 disp(['Đã lưu ảnh kết quả với tên: rotated_' num2str(degree) '_degrees.jpg']);
88 end

```

5.3.3 Kết quả của đoạn code



Hình 1: Kết quả xoay 180 độ



Hình 2: Kết quả xoay 45 độ

5.3.4 Nhận xét

Ưu điểm:

- Dễ hiểu, dễ hiện thực.
- Linh hoạt với mọi góc xoay.
- Không cần dùng thư viện xử lý ảnh nâng cao.

Nhược điểm:

- Có thể gây rối do làm tròn tọa độ
- Một số pixel trong ảnh mới không nhận được giá trị (nền đen).
- Không dùng nội suy, nên ảnh xoay có thể bị giảm chất lượng.

6 Lời kết

Trong báo cáo này, chúng em đã trình bày tổng quan lý thuyết về phép quay Givens – một công cụ tuyến tính hiệu quả trong việc khử phần tử dưới đường chéo chính của ma trận, và thực hiện phân rã QR ổn định về mặt số học. Bằng cách áp dụng vào xử lý hình ảnh, đặc biệt là trong việc nén dữ liệu, lọc nhiễu, tái tạo ảnh và chuẩn hóa cấu trúc hình học, phép quay Givens cho thấy tiềm năng to lớn trong việc cải thiện hiệu quả tính toán, đồng thời duy trì độ chính xác cao.

Thông qua thuật toán xây dựng xử lý ảnh, chúng em đã chứng minh được khả năng ứng dụng thực tiễn của phép quay Givens, đặc biệt là khi xử lý ảnh có độ phân giải lớn hoặc yêu cầu bảo toàn cấu trúc tuyến tính của dữ liệu. Về mặt tổng quát, phương pháp này có thể được tích hợp hiệu quả vào các hệ thống thị giác máy tính, đặc biệt trong bước tiền xử lý dữ liệu trước khi đưa vào mô hình học máy. Trong tương lai, đề tài có thể tiếp tục phát triển theo các hướng như:

- Tối ưu hóa tính toán thông qua xử lý song song hoặc phần cứng (GPU/FPGA).
- Kết hợp với các kỹ thuật hiện đại như CORDIC để tăng tốc tính toán lượng giác.
- Ứng dụng trong các lĩnh vực chuyên sâu như xử lý ảnh y tế, ảnh ba chiều, hoặc robot tự hành.

Ngoài ra, việc kết hợp phép quay Givens với các phép biến đổi khác như Householder hay SVD có thể mở ra các thuật toán lai hiệu quả hơn trong việc phân tích và tái cấu trúc dữ liệu hình ảnh phức tạp. Những hướng nghiên cứu này không chỉ góp phần nâng cao hiệu quả thuật toán trong xử lý ảnh, mà còn mở rộng tầm ảnh hưởng của đại số tuyến tính vào các lĩnh vực liên ngành như học sâu, điện toán hiệu năng cao và trí tuệ nhân tạo.

Nói tóm lại, phép quay Givens không chỉ là một công cụ mạnh mẽ trong không gian toán học thuần túy mà còn là một mắt xích quan trọng trong việc xây dựng cầu nối giữa lý thuyết và ứng dụng thực tiễn – đặc biệt là trong kỷ nguyên bùng nổ của dữ liệu hình ảnh và trí tuệ nhân tạo ngày nay.

7 Tài liệu tham khảo

Tài liệu

- [1] Đặng Văn Vinh. (2022). *Giáo trình Đại số tuyến tính*. Nhà xuất bản Đại học Quốc Gia TP.Hồ Chí Minh.
- [2] Yang, H., Patras, I. (2011). *Givens Rotation based Head Pose Estimation*. IEEE Transactions on Image Processing, 20(4), 1234-1245.
- [3] Szeliski, R. (2010). *Computer Vision: Algorithms and Applications*. Springer.
- [4] Golub, G. H., Van Loan, C. F. (2013). *Matrix Computations (4th ed.)*. Johns Hopkins University Press.
Dudgeon, D. E., Mersereau, R. M. (1990). *Multidimensional Digital Signal Processing*. Prentice Hall.
- [5] Qi, C. R., Su, H., Mo, K., Guibas, L. J. (2017). *PointNet: Deep Learning on Point Sets for 3D Classification and Segmentation*. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 652-660.
Esteves, C., Allen-Blanchette, C., Makadia, A., Daniilidis, K. (2018). *Learning $SO(3)$ -Equivariant Representations with Spherical CNNs*. Advances in Neural Information Processing Systems (NeurIPS), 31, 1017-1027.
- [6] Author, F., et al. (2024). *Modified CORDIC Algorithm for Givens Rotator*. Proceedings of the International Conference on Computational Science (ICCS 2024), 123-135.
- [7] Wikipedia contributors. (n.d.). *Givens rotation*. Wikipedia. Truy cập ngày 22 tháng 4 năm 2025. [Truy cập tại đây](#)
- [8] Trefethen, L. N., Bau, D. (1997). *Numerical Linear Algebra*. SIAM.
- [9] Watkins, D. S. (2002). *Fundamentals of Matrix Computations (2nd ed.)*. Wiley.
- [10] Hansen, P. C., Pereyra, V., Scherer, G. (2013). *Least Squares Data Fitting with Applications*. Johns Hopkins University Press.